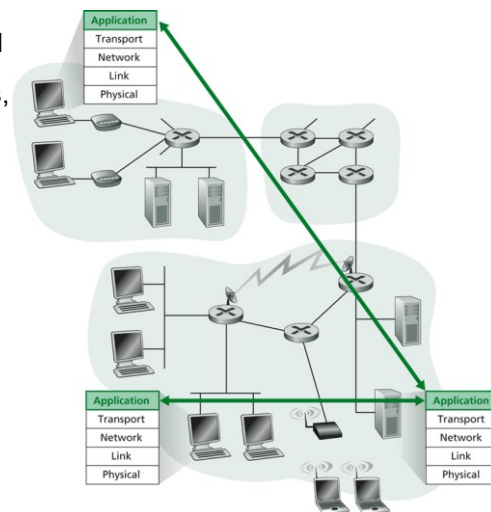


08 The Application Layer

08.01-08.02 Electronic mail, File transfer

The application layer

- No network applications would mean no reason to do networking, studying protocols, etc.
- Application programs are executed on terminal systems and communicate each other through a network
- Architectures: client-server, peer-to-peer
- Some examples:
 - E-mail
 - Web
 - Instant messaging
 - Remote login (telnet and ssh)
 - File transfer (ftp)
 - Internet telephony
 - Real time videoconference
 - ...



Copyright © 2005 Pearson Addison Wesley All rights reserved

1 ♦ Communication for a network application takes place between end systems at the application layer.

Electronic Mail

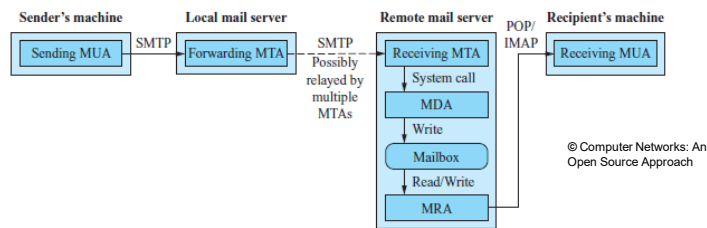
- Probably the most popular application from the beginning of the Internet
- Asynchronous communication:
 - Messages are sent/read when user wants
 - Advantages: speed, ease of use, cost
 - Mailing list: more destination can be reached simultaneously
 - Attachments

E-Mail Addressing

- Each e-mail user can be reached by his own e-mail address, with a format defined as
 - *user@{host.}network*.
- An e-mail address consists of three parts. The first part and the second part identify the username and the mail server of the recipient, respectively.
- The third part tells the network or domain where the mail server is located.
- Note that the second part is often omitted for simplicity because the mail server of a domain can be looked up from DNS's mail exchanger (MX) resource records

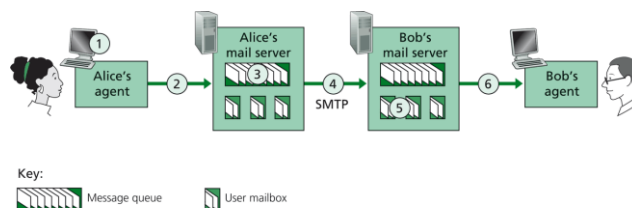
E-MAIL main components

- Mail User agent
 - They allow users to read, answer, forward, compose, save messages (Outlook, Mozilla Mail, Thunderbird, pine, elm)
- Mail Transfer agent
 - MTAs such as sendmail, qmail, and postfix for the UNIX platforms and Ms Exchange Server for windows platforms are used to accept messages from MUAs through Simple Mail Transfer Protocol (SMTP) and pass them directly to the MTA at the remote mail server or to the intermediate MTA for relaying.
- Mail Delivery agent
 - The MDA at the remote mail server then obtains messages from the receiving MTA and writes them into the recipient's mailbox for later retrieval
- Mail Retrieval agent
 - An MRA is used to fetch messages from a mailbox on a server and then passes them to an MUA through Post Office Protocol (POP) or Internet Message Access Protocol (IMAP).



SMTP (Simple Mail Transfer Protocol)

- It is an old protocol (older than HTTP)
- Headers and bodies are treated as 7 bit ASCII (multimedia binary data need to be encoded and decoded)
- Alice calls its user agent, compose message and tells user agent to send it
- User address format: user@dns-address (the server is identified through DNS)
- Alice's user agents sends message to its mail server (queued)
- SMTP client of Alice's mail server opens a TCP connection to SMTP Bob's mail server
- After SMTP handshaking SMTP client sends message on TCP connection
- SMTP server (Bob's mail server) receives and puts message in Bob's mailbox
- When Bob wants, he calls his user agent to read message
- Usually no intermediate mail servers are used



Message Transfer: SMTP

- Example of messages exchanged between an **SMTP client (C)** and an **SMTP server (S)**
 - Ascii text is sent on TCP socket
 - Client commands:
 - HELO, MAIL FROM, RCPT TO, DATA, QUIT
 - Each message ends with CRLF
 - Server answers:
 - A code number + an optional comment
 - SMTP uses persistent connections (if sender mail server has more messages to send to the same receiver mail server → uses the same TCP connection)
- ```
S: 220 abcd.edu
C: HELO xyz.edu
S: 250 Hello xyz.edu, pleased to meet you
C: MAIL FROM: alice@xyz.edu
S: 250 alice@xyz.edu ... Sender ok
C: RCPT TO: <bob@abcd.edu>
S: 250 bob@abcd.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Hello my old friend.
C: How are you?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 abcd.edu closing connection
```

# Message Formats – RFC 822

- Before E-mail message body a header is placed
- Header contains service information
- RFC 822 specifies header format and semantic
- Header and body are separated by a white line (CR LF)
- Header: ascii keywords: value
  - **From:** alice@xyz.edu
  - **To:** bob@abcd.edu
  - **Subject:** How are you

# Message Formats – RFC 822

- RFC 822 header fields related to message transport
- Some fields used in the RFC 822 message header.

| Header       | Meaning                                           |
|--------------|---------------------------------------------------|
| To:          | E-mail address(es) of primary recipient(s)        |
| Cc:          | E-mail address(es) of secondary recipient(s)      |
| Bcc:         | E-mail address(es) for blind carbon copies        |
| From:        | Person or people who created the message          |
| Sender:      | E-mail address of the actual sender               |
| Received:    | Line added by each transfer agent along the route |
| Return-Path: | Can be used to identify a path back to the sender |

| Header       | Meaning                                               |
|--------------|-------------------------------------------------------|
| Date:        | The date and time the message was sent                |
| Reply-To:    | E-mail address to which replies should be sent        |
| Message-Id:  | Unique number for referencing this message later      |
| In-Reply-To: | Message-Id of the message to which this is a reply    |
| References:  | Other relevant Message-Ids                            |
| Keywords:    | User-chosen keywords                                  |
| Subject:     | Short summary of the message for the one-line display |

# MIME – Multipurpose Internet Mail Extensions

- Problems with international languages:
  - Languages with accents (French, German, Italian).
  - Languages in non-Latin alphabets (Hebrew, Russian).
  - Languages without alphabets (Chinese, Japanese).
- Messages not containing text at all (audio or images)
- To manage these situation RFC 2045 and 2046 defined extension headers MIME
- 2 key header to support multimedia are:
  - **Content-type:**
    - It is used by user agent receiver to properly process message body (e.g. JPEG decompression)
  - **Content-Transfer-Encoding:**
    - It is used for conversion of message body in original format (non ASCII messages must be converted to ASCII for SMTP)

## MIME extension

- Example: Alice sends a JPEG picture to Bob
- Alice's user agent creates a message like:

**From:** alice@xyz.edu

**To:** bob@abcd.edu

**Subject:** Picture of new house

**MIME-Version:** 1.0

**Content-Transfer-Encoding:** base 64

**Content-type:** image/jpeg

(base 64 encoded data ...

.....

..... base 64 encoded data)

## MIME extension

- In the previous example the sender's user agent encoding
- Other MIME standard encoding techniques (to encode data into 7 bit ASCII) are possible
  - E.g. Quoted-printable content transfer encoding is usually adopted for ASCII 8 bit → ASCII 7 bit conversion
- When Bob's user agent reads MIME message:
  - It correctly decodes the message body (e.g. base 64)
  - The Content-Type: header tells Bob's user agent that message body should be JPEG decompressed

# MIME

RFC 822 headers added by MIME.

| Header                     | Meaning                                              |
|----------------------------|------------------------------------------------------|
| MIME-Version:              | Identifies the MIME version                          |
| Content-Description:       | Human-readable string telling what is in the message |
| Content-Id:                | Unique identifier                                    |
| Content-Transfer-Encoding: | How the body is wrapped for transmission             |
| Content-Type:              | Type and format of the content                       |

# MIME

The MIME types and subtypes defined in RFC 2045.

| Type        | Subtype       | Description                                 |
|-------------|---------------|---------------------------------------------|
| Text        | Plain         | Unformatted text                            |
|             | Enriched      | Text including simple formatting commands   |
| Image       | Gif           | Still picture in GIF format                 |
|             | Jpeg          | Still picture in JPEG format                |
| Audio       | Basic         | Audible sound                               |
| Video       | Mpeg          | Movie in MPEG format                        |
| Application | Octet-stream  | An uninterpreted byte sequence              |
|             | Postscript    | A printable document in PostScript          |
| Message     | Rfc822        | A MIME RFC 822 message                      |
|             | Partial       | Message has been split for transmission     |
|             | External-body | Message itself must be fetched over the net |
| Multipart   | Mixed         | Independent parts in the specified order    |
|             | Alternative   | Same message in different formats           |
|             | Parallel      | Parts must be viewed simultaneously         |
|             | Digest        | Each part is a complete RFC 822 message     |

# MIME

A multipart message containing enriched and audio alternatives.

From: elinor@abcd.com  
To: carolyn@xyz.com  
MIME-Version: 1.0  
Message-Id: <0704760941.AA00747@abcd.com>  
Content-Type: multipart/alternative; boundary=qwertyuiopasdfghjklzxcvbnm  
Subject: Earth orbits sun integral number of times

This is the preamble. The user agent ignores it. Have a nice day.

--qwertyuiopasdfghjklzxcvbnm  
Content-Type: text/enriched

Happy birthday to you  
Happy birthday to you  
Happy birthday dear <bold> Carolyn </bold>  
Happy birthday to you

--qwertyuiopasdfghjklzxcvbnm  
Content-Type: message/external-body;  
access-type="anon-ftp";  
site="bicycle.abcd.com";  
directory="pub";  
name="birthday.snd"

content-type: audio/basic  
content-transfer-encoding: base64  
--qwertyuiopasdfghjklzxcvbnm--

# SSL vs TLS vs STARTTLS

- There's often quite a bit of confusion around the different terms **SSL**, **TLS** and **STARTTLS**.
- **SSL** and **TLS** both provide a way to encrypt a communication channel between two computers (e.g. your computer and our server). TLS is the successor to SSL and the terms SSL and TLS are used interchangeably unless you're referring to a specific version of the protocol.
- **STARTTLS** is a way to take an existing insecure connection and upgrade it to a secure connection using SSL/TLS. Note that despite having TLS in the name, STARTTLS doesn't mean you have to use TLS, you can use SSL.

## TLS vs STARTTLS naming problem

- One significant complicating factor is that some email software **incorrectly** uses the term TLS when they should have used STARTTLS.
- Some old versions of Thunderbird in particular used "TLS" to mean "enforce use of STARTTLS to upgrade the connection, and fail if STARTTLS is not supported" and "TLS, if available" to mean "use STARTTLS to upgrade the connection if the server advertises support for it, otherwise just use an insecure connection".

## SSL/TLS vs plaintext/STARTTLS port numbers

- The above is particularly problematic when combined with having to configure a port number for each protocol.
- To add security to some existing protocols (e.g. IMAP, POP, etc.), it was decided to just add SSL/TLS encryption as a layer underneath the existing protocol. However, to distinguish that software should talk the SSL/TLS encrypted version of the protocol rather than the plaintext one, a different port number was used for each protocol. So you have:
  - IMAP uses port 143, but SSL/TLS encrypted IMAP uses port 993.
  - POP uses port 110, but SSL/TLS encrypted POP uses port 995.
  - SMTP uses port 25, but SSL/TLS encrypted SMTP uses port 465.

## SSL/TLS vs plaintext/STARTTLS port numbers

- At some point, it was decided that having 2 ports for every protocol was wasteful, and instead you should have 1 port that starts off as plaintext, but the client can upgrade the connection to an SSL/TLS encrypted one. This is what STARTTLS was created to do.
- There were a few problems with this though. There was already existing software that used the alternate port numbers with pure SSL/TLS connections. Client software can be very long lived, so you can't just disable the encrypted ports until all software has been upgraded.

## SSL/TLS vs plaintext/STARTTLS port numbers

- Mechanisms were added to each protocol to tell clients that the plaintext protocol supported upgrading to SSL/TLS (i.e. STARTTLS), and that they should not attempt to log in without doing the STARTTLS upgrade. This created two unfortunate situations:
  - Some software just ignored the "login disabled until upgraded" announcement and just tried to log in anyway, sending the username and password over plaintext. Even if the server then rejected the login, the details had already been sent over the Internet in plaintext.
  - Other software saw the "login disabled until upgraded" announcement, but then wouldn't upgrade the connection automatically, and thus reported login errors back to the user, which caused confusion about what was wrong.

## SSL/TLS vs plaintext/STARTTLS port numbers

- Both of these problems resulted in significant compatibility issues with existing clients, and so most system administrators continued to just use plaintext connections on one port number, and encrypted connections on a separate port number.
- This has now basically become the de facto standard that everyone uses. IMAP SSL/TLS encrypted over port 993 or POP SSL/TLS encrypted over port 995. Many sites now disable plain IMAP (port 143) and plain POP (port 110) altogether so people **must** use an SSL/TLS encrypted connection. By disabling ports 143 and 110, this removes completely STARTTLS as even an option for IMAP/POP connections.

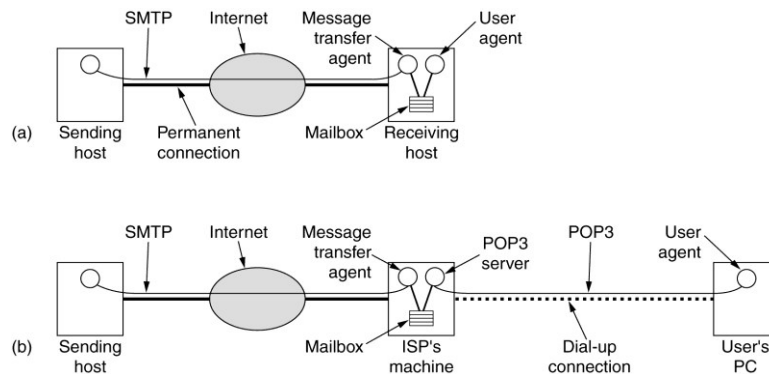
## SMTP STARTTLS as an exception

- The one real exception to the above is SMTP. However that's for a different reason again. Most email software (mail user agent) used SMTP on port 25 to submit messages to the email server for onward transmission to the destination (mail transfer agent). However, SMTP was originally designed for transfer, not submission. So yet another port (587) was defined for message submission.
- Although port 587 doesn't mandate requiring STARTTLS, the use of port 587 became popular around the same time as the realisation that SSL/TLS encryption of communications between clients and servers was an important security and privacy issue and encryption extensions were being defined for SMTP. So shortly after port 465 was defined, it was revoked with the expectation that clients would move to using STARTTLS over port 587

## SMTP STARTTLS as an exception

- The result is that in most cases, systems that offer message submission over port 587 **require** clients to use STARTLS to upgrade the connection and also require a username and password to authenticate. There has been an added benefit to this approach as well. By moving users away from using port 25 for email submission, ISPs are now able to block outgoing port 25 connections from users' computers, which were a significant source of spam due to infection with spam-sending viruses.
- Unfortunately the downside of changing port numbers is that a number of email clients were made which only supported SSL/TLS over port 465 and not STARTTLS on 587. Clients are often very long lived, and so removing port 465 wasn't an option for many sites without annoying customers.
- Additionally, because port 465 was advertised as an option, many users with email clients that support both STARTTLS on 587 and SSL/TLS on 465 set them up to use 465 instead of 587. This makes it even harder to remove support for port 465, since lots of users have their email clients set up to use it.

## Final Delivery



- (a) Sending and reading mail when the receiver has a permanent Internet connection and the user agent runs on the same machine as the message transfer agent. (b) Reading e-mail when the receiver has a dial-up connection to an ISP.

## POP3

- User agent (client) opens a TCP connection to the mail server (server) on port 110
- Once TCP connection is established:
  - Authorization:
    - User agent sends username+password for authentication
  - Commands:
    - user <username>
    - pass <password>
  - Transaction
    - User agent takes messages and marks those to be deleted
    - Some commands:
      - list, retr, dele, quit
  - Update
    - After the quit command from client (that closes the POP3 session), the mail server deletes marked messages
- User agent sends messages
- Server reacts:
  - +OK (possibly followed by some data)
  - -ERR

## POP3

Using POP3 to fetch three messages.

```
S: +OK POP3 server ready
C: USER carolyn
S: +OK
C: PASS vegetables
S: +OK login successful
C: LIST
S: 1 2505
S: 2 14302
S: 3 8122
S: .
C: RETR 1
S: (sends message 1)
C: DELE 1
C: RETR 2
S: (sends message 2)
C: DELE 2
C: RETR 3
S: (sends message 3)
C: DELE 3
C: QUIT
S: +OK POP3 server disconnecting
```

# IMAP

- IMAP4, first defined in RFC 1730 and later obsoleted by RFC 2060 and 3501, is proposed as a replacement for the POP3 protocol.
- An IMAP4 session progresses through three stages: the establishment of a client/server connection, an initial greeting from the server, and client/server interactions.
- An IMAP4 server can be in four states described as follows:
  1. **Non-authenticated** : When a connection is established between the IMAP4 server and the client, the server enters the non-authenticated state. The client must supply authentication credentials before most commands can be permitted.
  2. **Authenticated** : When a pre-authenticated connection starts, the server enters the authenticated state when acceptable authentication credentials have been provided or after an error in mailbox selection. In the authenticated state, the client must select a mailbox to access before commands that affect messages can be permitted.
  3. **Selected** : When a mailbox has been successfully selected, the server enters the selected state. In this state, a mailbox has been selected to access.
  4. **Logout** : When the client asks to exit the server, the server enters the logout state. At this time, the server will close the connection.

# IMAP Command

- tag command argument(s)
  - tag, either a "." or a text string that can be sequentialized (a0001, a0002, a0003....); if only a single connection use ".", if multiple connections use text string (this allows matching commands with responses).

# IMAP - Commands

- login
- list
- status
- examine
- select
- create, delete, rename
- fetch
- store
- close
- expunge
- copy
- idle
- lsub, subscribe, unsubscribe
- logout
- capability, getquotaroot, getacl

# IMAP - Commands

- login
  - userid@address
  - password
  - example
    - . login steflik@binghamton.edu xyz123  
(if not using ssl this goes as plain text just like pop)
  - response – server should acknowledge with OK

# IMAP Commands

- logout command
  - no arguments
- The command is sent to the server, the server replies with a BYE message followed by an OK message and closes the connection.

# IMAP - Commands

- list
  - retrieves a list of the mailboxes/folders
  - arguments :
    - reference name:
    - mailbox name w/possible wildcards
  - returns an untagged list of the mailboxes/folders along with a separator char and an indication of hierarchy followed by a tagged OK
  - for details refer to rfc 1730

# IMAP Commands

- status command
  - arguments:  
mailbox/folder  
(space delimited list of flags)
  - return info you asked for; untagged list
  - **EX:** a006 status inbox (messages uidnext)  
\* STATUS "inbox" (MESSAGES 404 UIDNEXT 1001)  
a006 OK Success

# IMAP Commands

- examine & select commands
  - arguments:  
mailbox/folder
  - returns: Flags information, how many of  
messages each flag type are in the folder;  
then allows access to the messages in the  
folder
  - examine allows read-only  
select allows read-write access

# IMAP Commands

- create, delete and rename commands
- create newfoldernane – creates a new folder
- delete foldernane – deletes the named folder
- rename oldname newname – renames the folder
- foldernames must be fully qualified using the separator char from the info returned by the list command

# IMAP Commands

- fetch command – used to actually access e-mails
- arguments:
  - message number or range of numbers  
( 1 , 1:2 , 1:last)
  - what it is you want to retrieve  
( fast, all, text, rfc822.header....see rfc)



# IMAP Commands

- store – lets you set the flags for messages
- arguments:
  - message number/range
  - flags to be set  
(\Answered \Flagged \Draft \Deleted  
\Seen hasatt Junk )



# IMAP Commands

- close & expunge commands – used to permanently delete a message(s) in the current folder that has the \Deleted flag set.



# IMAP Commands

- copy command – copy a message or range of messages to another folder then delete the originals
- arguments:
  - message number or range
  - target folder
- ex: copy 1:3 linux.debian

# IMAP Commands

- idle command – lets you monitor a folder until something new is added to it

# IMAP Commands

- capability command – untagged list of the servers capabilities
- getquotaroot – returns the amount of space you are using and how much is available
  - ex: getquitaroot inbox
- getacl command – returns the access control list for the specified folder (l,r,s,w,l,p,c,d,a)

# Webmail

- Users can access their e-mail by means of a web browser
- The web browser is the user agent
- User communicates with its remote mailbox using HTTP
  - Messages are sent from mail server to user destination browser by HTTP (instead of POP3/IMAP)
  - Senders push messages to mail servers by means of browser using HTTP (instead of SMTP)
- Many implementations use IMAP server to provide folder functionalities (messages are accessed thanks to script executed on HTTP server that use IMAP protocol to communicate with IMAP server)

## POP3 – IMAP differences

| Feature                        | POP3      | IMAP             |
|--------------------------------|-----------|------------------|
| Where is protocol defined?     | RFC 1939  | RFC 2060         |
| Which TCP port is used?        | 110       | 143              |
| Where is e-mail stored?        | User's PC | Server           |
| Where is e-mail read?          | Off-line  | On-line          |
| Connect time required?         | Little    | Much             |
| Use of server resources?       | Minimal   | Extensive        |
| Multiple mailboxes?            | No        | Yes              |
| Who backs up mailboxes?        | User      | ISP              |
| Good for mobile users?         | No        | Yes              |
| User control over downloading? | Little    | Great            |
| Partial message downloads?     | No        | Yes              |
| Are disk quotas a problem?     | No        | Could be in time |
| Simple to implement?           | Yes       | No               |
| Widespread support?            | Yes       | Growing          |

- However the main difference between IMAP4 and POP3 is that IMAP4 allows messages to be stored and manipulated on the mail system while POP3 only allows users to download their messages and store and manipulate messages on the users machines.

## Email best practices

- Some email best practices to have your emails delivered are:
  - Reputation
  - Hosting
  - Ip addresses and sending volume
  - DNS
  - Authentication

# Reputation

- One of the most important assets you have in the email world (much like the real world) is your reputation.
- A good analogy for your email reputation is your personal credit score. Obviously, a bad reputation will hurt you. However, not having a reputation will also hurt you.
- Beyond making sure that the infrastructure is properly set up, you have to answer some important questions:
  - Are emails being delivered and if not, why?
  - Is a recipient ESP throttling your traffic and why?
  - Are messages bouncing due to incorrect domains or stale addresses?
  - Are recipients unsubscribing or complaining of spam?
  - Are recipients engaging with your emails by opening them and/or clicking on links?

# Hosting

- A brief note on hosting since hosting technology is changing so quickly, and large, virtual cloud environments are generally not the best for email, reasons:
  - The IP address should be static so that your domain(s) & IP address(es) build a reputation together.
  - The IP address and surrounding IP addresses should have a good reputation.
  - Mail Transfer Agents should ideally be on real (non-virtual) machines, optimized for I/O.

## Ip addresses and sending volume

- If you are sending a lot of email (greater than 50k per week), it is a good idea to have a dedicated IP in order to isolate your reputation.
- If you are sharing your IP, you are sharing your reputation with those other senders.
- The other thing to consider is using separate IPs for your bulk and transactional mail if you are sending high volumes of email.

## DNS

- Your email reputation is not only tied to your IP, but your domain name as well.
- It is a good idea to have separate domains or subdomains for your marketing, transactional and corporate mail.
- You should also make sure that you are using a well regarded DNS provider and that you publish all of your contact information in the WHOIS record.

# Authentication

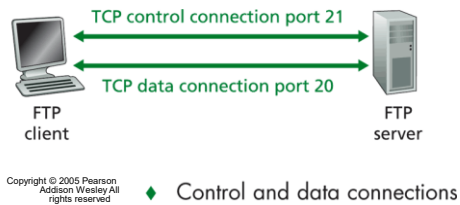
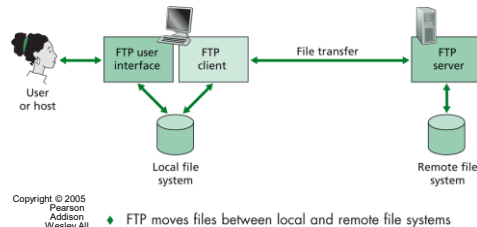
- It is very important that you are using the appropriate authentication methods with your email, if not your email can be recognized as spam and it will be filtered or just dropped.
- The common types of authentication are:
  - SPF
  - DKIM
  - DomainKeys
  - SenderID

# FTP: file transfer protocol

- The File Transfer Protocol (FTP) was designed to allow users to efficiently and reliably transfer files from one host to another over the Internet.
- FTP provides data replication, which enables a larger scale of data backup.
- FTP was first proposed in RFC 172 in 1971 and finally standardized in RFC 959 in 1985. RFC 3659 in 2007 is the latest update on FTP extensions
- FTP offers two kinds of accesses:
  - *Authenticated*
  - *anonymous*.
- *The former* requires an account /password pair for user authentication, while the latter is usually unrestricted

# FTP: file transfer protocol

- Remote file system access
- ftp client sets a TCP connection with remote ftp server
- Username & password are sent as ftp commands
- Then files can be transferred to/from server
- Differently from http it is an *out-of-band* protocol: 2 parallel TCP connections, **data** (non persistent) and **control** (persistent)



## FTP: Access cmds

**Table 21.1** Access commands

| Command | Argument(s)           | Description                |
|---------|-----------------------|----------------------------|
| USER    | User id               | User information           |
| PASS    | User password         | Password                   |
| ACCT    | Account to be charged | Account information        |
| REIN    |                       | Reinitialize               |
| QUIT    |                       | Log out of the system      |
| ABOR    |                       | Abort the previous command |

# FTP: Mgmt cmds

**Table 21.2** *File management commands*

| Command      | Argument(s)      | Description                                     |
|--------------|------------------|-------------------------------------------------|
| <b>CWD</b>   | Directory name   | Change to another directory                     |
| <b>CDUP</b>  |                  | Change to parent directory                      |
| <b>DELE</b>  | File name        | Delete a file                                   |
| <b>LIST</b>  | Directory name   | List subdirectories or files                    |
| <b>NLIST</b> | Directory name   | List subdirectories or files without attributes |
| <b>MKD</b>   | Directory name   | Create a new directory                          |
| <b>PWD</b>   |                  | Display name of current directory               |
| <b>RMD</b>   | Directory name   | Delete a directory                              |
| <b>RNFR</b>  | File name (old)  | Identify a file to be renamed                   |
| <b>RNTD</b>  | File name (new)  | Rename the file                                 |
| <b>SMNT</b>  | File system name | Mount a file system                             |

# FTP: Other cmds

**Table 21.3** *Data formatting commands*

| Command     | Argument(s)                                                                                      | Description                 |
|-------------|--------------------------------------------------------------------------------------------------|-----------------------------|
| <b>TYPE</b> | <b>A</b> (ASCII), <b>E</b> (EBCDIC), <b>I</b> (Image), <b>N</b> (Nonprint), or <b>T</b> (TELNET) | Define file type            |
| <b>STRU</b> | <b>F</b> (File), <b>R</b> (Record), or <b>P</b> (Page)                                           | Define organization of data |
| <b>MODE</b> | <b>S</b> (Stream), <b>B</b> (Block), or <b>C</b> (Compressed)                                    | Define transmission mode    |

**Table 21.4** *Port defining commands*

| Command     | Argument(s)        | Description           |
|-------------|--------------------|-----------------------|
| <b>PORT</b> | 6-digit identifier | Client chooses a port |
| <b>PASV</b> |                    | Server chooses a port |



# FTP: Responses

Table 21.7 Responses

| Code                               | Description                                                        |
|------------------------------------|--------------------------------------------------------------------|
| <b>Positive Preliminary Reply</b>  |                                                                    |
| 120                                | Service will be ready shortly                                      |
| 125                                | Data connection open; data transfer will start shortly             |
| 150                                | File status is OK; data connection will be open shortly            |
| <b>Positive Completion Reply</b>   |                                                                    |
| 200                                | Command OK                                                         |
| 211                                | System status or help reply                                        |
| 212                                | Directory status                                                   |
| 213                                | File status                                                        |
| 214                                | Help message                                                       |
| 215                                | Naming the system type (operating system)                          |
| 220                                | Service ready                                                      |
| 221                                | Service closing                                                    |
| 225                                | Data connection open                                               |
| 226                                | Closing data connection                                            |
| 227                                | Entering passive mode; server sends its IP address and port number |
| 230                                | User login OK                                                      |
| 250                                | Request file action OK                                             |
| <b>Positive Intermediate Reply</b> |                                                                    |
| 331                                | User name OK; password is needed                                   |
| 332                                | Need account for logging                                           |
| 350                                | The file action is pending; more information needed                |



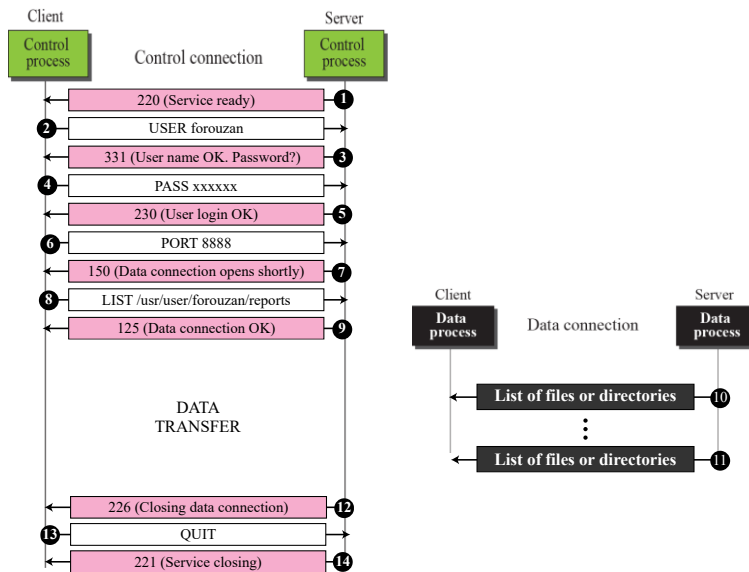
# FTP: Responses

Table 21.7 Responses (continued)

| Code                                       | Description                                           |
|--------------------------------------------|-------------------------------------------------------|
| <b>Transient Negative Completion Reply</b> |                                                       |
| 425                                        | Cannot open data connection                           |
| 426                                        | Connection closed; transfer aborted                   |
| 450                                        | File action not taken; file not available             |
| 451                                        | Action aborted; local error                           |
| 452                                        | Action aborted; insufficient storage                  |
| <b>Permanent Negative Completion Reply</b> |                                                       |
| 500                                        | Syntax error; unrecognized command                    |
| 501                                        | Syntax error in parameters or arguments               |
| 502                                        | Command not implemented                               |
| 503                                        | Bad sequence of commands                              |
| 504                                        | Command parameter not implemented                     |
| 530                                        | User not logged in                                    |
| 532                                        | Need account for storing file                         |
| 550                                        | Action is not done; file unavailable                  |
| 552                                        | Requested action aborted; exceeded storage allocation |
| 553                                        | Requested action not taken; file name not allowed     |



# FTP: Sample flow



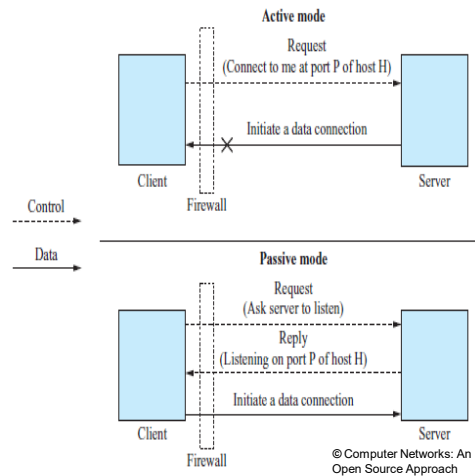
# FTP Session Sample

- The colored lines show the responses from the server control connection; the black lines show the commands sent by the client. The lines in white with black background show data transfer.

```
$ ftp voyager.deanza.fhda.edu
Connected to voyager.deanza.fhda.edu.
220 (vsFTPd 1.2.1)
530 Please login with USER and PASS.
Name (voyager.deanza.fhda.edu:forouzan): forouzan
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls reports
227 Entering Passive Mode (153,18,17,11,238,169)
150 Here comes the directory listing.
drwxr-xr-x 2 3027 411 4096 Sep 24 2002 business
drwxr-xr-x 2 3027 411 4096 Sep 24 2002 personal
drwxr-xr-x 2 3027 411 4096 Sep 24 2002 school
226 Directory send OK.
ftp> quit
221 Goodbye.
```

# FTP: Active Mode vs. Passive Mode

- **Active mode:** the control connection is initiated by the client while the data connection is initiated by the server
- **Passive mode:** both connections are initiated by the client



## TFTP (rfc 783)

- There are occasions when we need to simply copy a file without the need for all of the features of the FTP protocol. For example, when a diskless workstation or a router is booted, we need to download the bootstrap and configuration files.
- Here we do not need all of the sophistication provided in FTP. We just need a protocol that quickly copies the files.

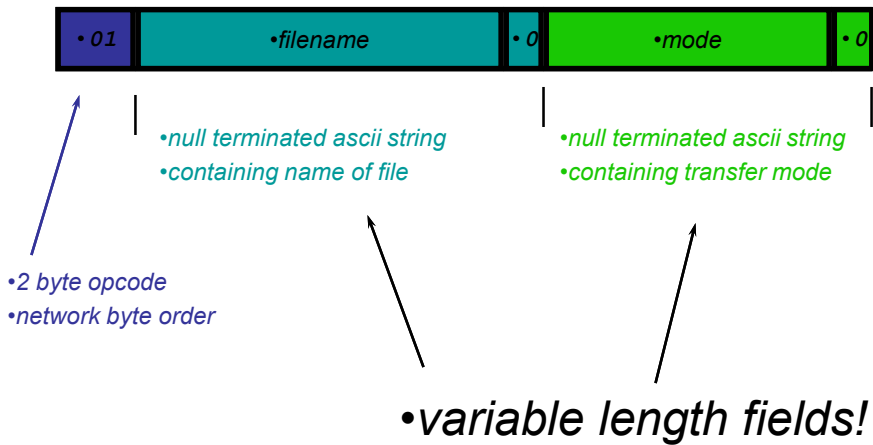
# TFTP Usage and Design

- Transfer files between processes.
- Minimal overhead (no security)
- Designed for UDP, although could be used with many transport protocols.
- Easy to implement
- Small - possible to include in firmware
- Often uses to bootstrap workstations and network devices.

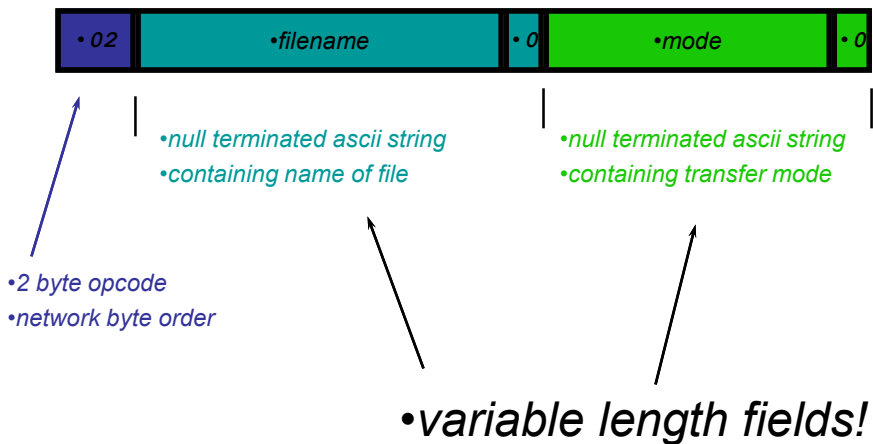
# TFTP Protocol

- 5 message types
  - Read request
  - Write request
  - Data
  - ACK (acknowledgment)
  - Error

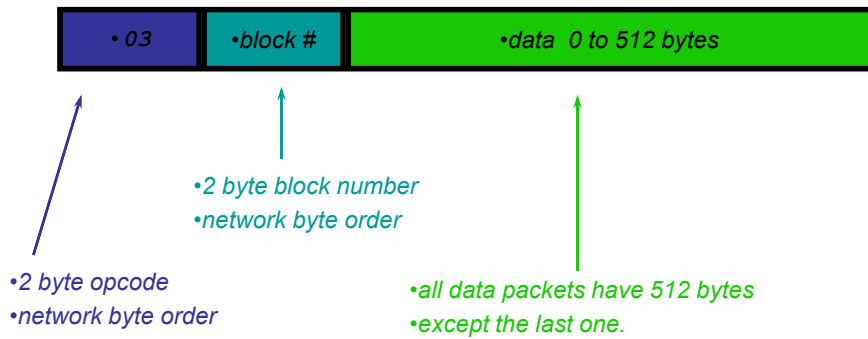
# TFTP: Read Request



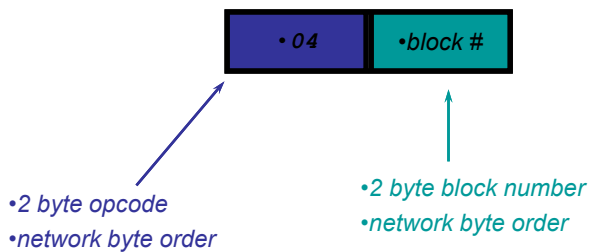
# TFTP: Write Request



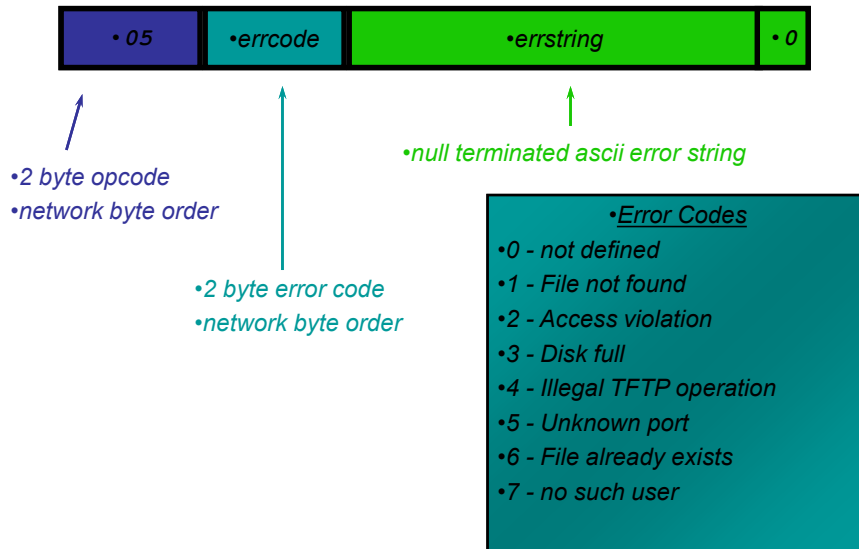
# TFTP: DATA packet



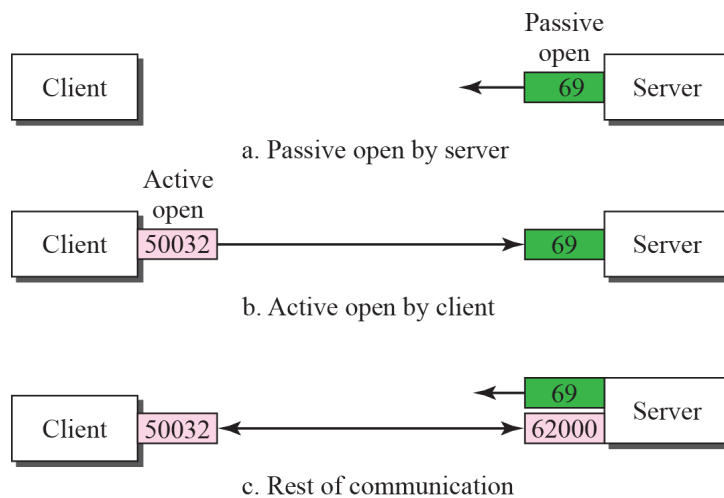
# TFTP: Ack



# TFTP: Error packet



# TFTP: Active/Passive mode



# TFTP Sample

